

EE446 Lab 6

2268225 Momoka Sakamoto

1. Raw inputs:
accX, accY, accZ
2. NN classifier features
accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, accY RMS
3. NN classifier outputs:
Nominal, off
4. Anomaly detector features:
accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, accY RMS

5. Deployment evidence:

demo2.6ohjq...	demo2	Today, 21:4...	10s	⋮
demo1.6ohjp...	demo1	Today, 21:4...	10s	⋮
nominal.2fc6...	nominal	Sep 13 202...	3s	⋮
off.2fc60g5p	off	Sep 13 202...	3s	⋮
off.2fc604h5	off	Sep 13 202...	3s	⋮
off.2fc5vr7r	off	Sep 13 202...	3s	⋮
nominal.2fc5...	nominal	Sep 13 202...	3s	⋮
off.2fc5tt13	off	Sep 13 202...	3s	⋮
off.2fc4bc2g	off	Sep 13 202...	3s	⋮
nominal.2fc4...	nominal	Sep 13 202...	3s	⋮
off.2fc2j1ck	off	Sep 13 202...	3s	⋮
off.2fc2ilc1	off	Sep 13 202...	3s	⋮

<123456>

10000

Sensor

Inertial (Accelerometer / Gyroscope) ▾

Frequency

100Hz ▾

Start sampling

RAW DATA

demo2.6ohjqc86

0ms1841ms3683ms5525ms7367ms9209ms

accX

accY

accZ

gyrX

gyrY

gyrZ

Metadata

? +

No metadata.

```

[WS ] Device "nano" is now connected to project "Fan Monitoring - Advanced Anomaly Detection". To c
onnect to another project, run `edge-impulse-daemon --clean`.
[WS ] Go to https://studio.edgeimpulse.com/studio/997978/acquisition/training to build your machine
learning model!

[WS ] Incoming sampling request {
  path: '/api/training/data',
  label: 'demo1',
  length: 10000,
  interval: 10,
  hmacKey: '216ebd2855b0a7f760be36ae50da6e5f',
  sensor: 'Inertial (Accelerometer / Gyroscope)'
}
[SER] Configured upload settings
[SER] Sampling started

[SER] Sampling done
[SER] Device not connected to WiFi directly, reading from buffer (bytes 0 - 28434, expecting to rea
d ~37912 bytes...
[SER] Reading 28% complete...
[SER] Reading 59% complete...
[SER] Reading 90% complete...
[SER] Reading from buffer OK
[SER] File is 28434 bytes after decoding
[SER] Uploading to https://ingestion.edgeimpulse.com/api/training/data
[SER] Uploading to https://ingestion.edgeimpulse.com/api/training/data OK

[WS ] Incoming sampling request {
  path: '/api/training/data',
  label: 'demo2',
  length: 10000,
  interval: 10,
  hmacKey: '216ebd2855b0a7f760be36ae50da6e5f',
  sensor: 'Inertial (Accelerometer / Gyroscope)'
}
[SER] Configured upload settings
[SER] Sampling started
[SER] Sampling done
[SER] Device not connected to WiFi directly, reading from buffer (bytes 0 - 30884, expecting to rea
d ~41178 bytes...
[SER] Reading 26% complete...
[SER] Reading 54% complete...
[SER] Reading 83% complete...
[SER] Reading from buffer OK
[SER] File is 30884 bytes after decoding
[SER] Uploading to https://ingestion.edgeimpulse.com/api/training/data
[SER] Uploading to https://ingestion.edgeimpulse.com/api/training/data OK

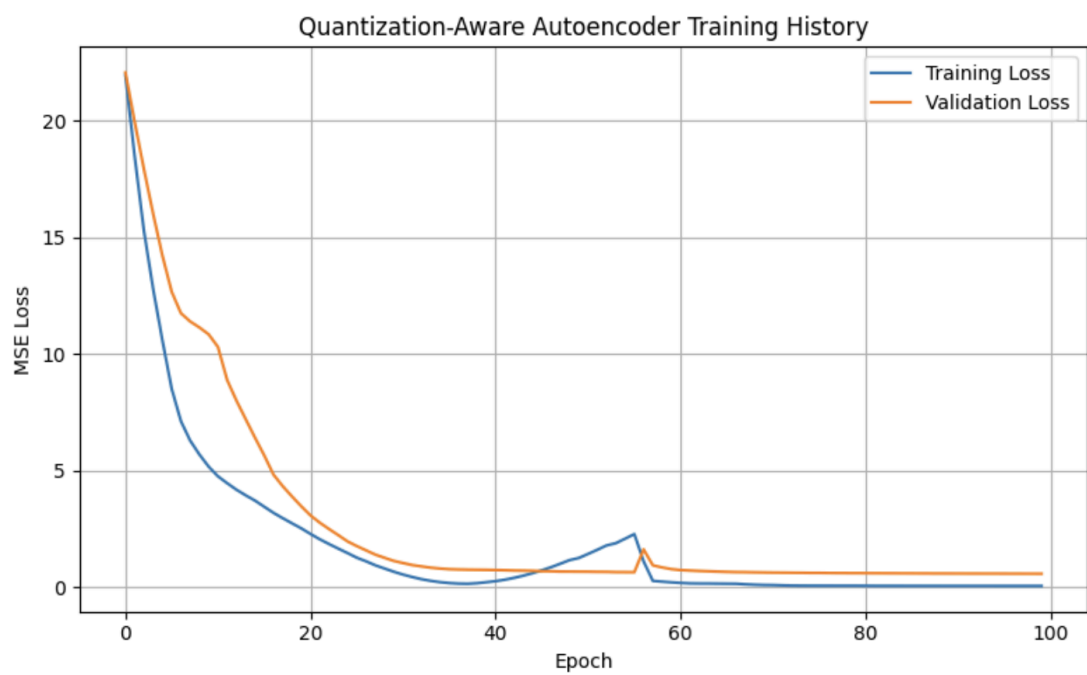
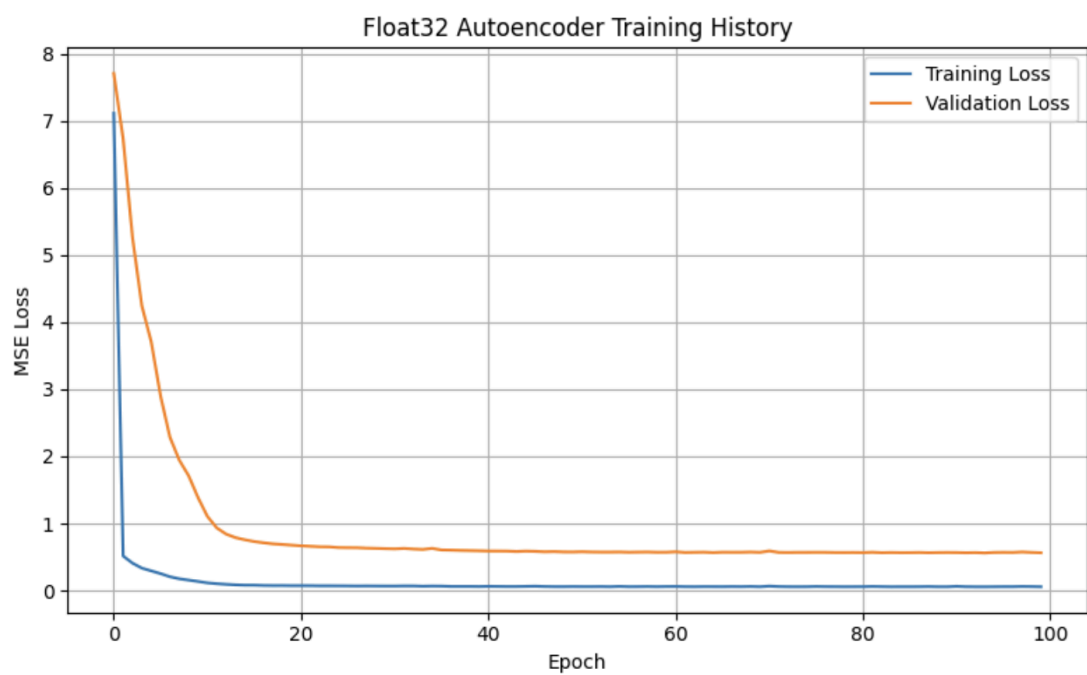
```

6. Short interpretation:

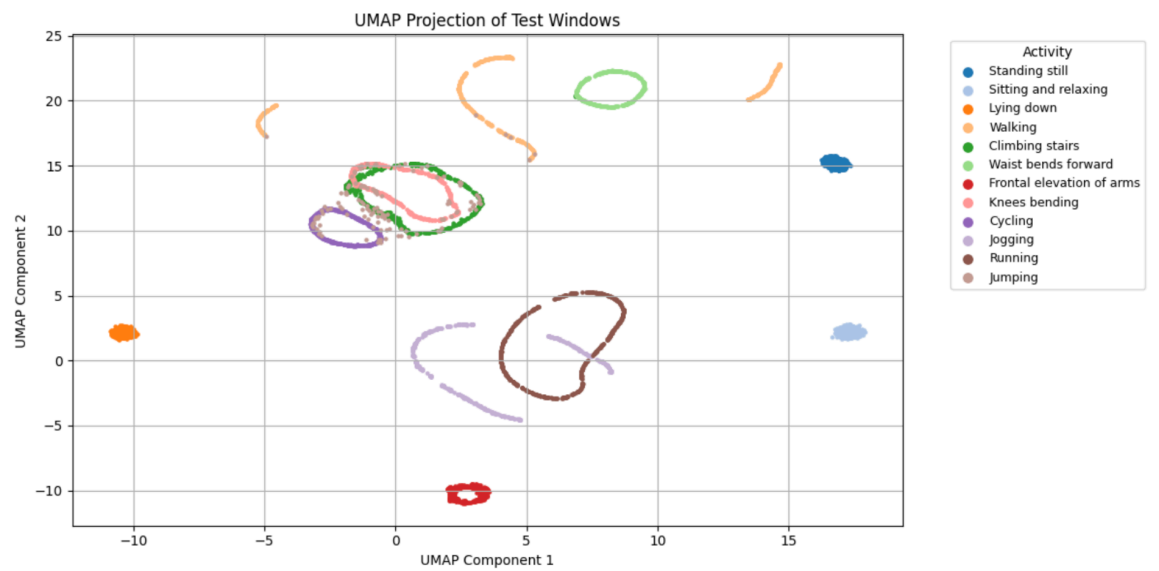
The deployed model successfully performed real-time classification and anomaly detection on the Arduino Nano 33 BLE Sense. The model was able to distinguish between nominal and off fan operating conditions while generating anomaly scores for unusual behavior.

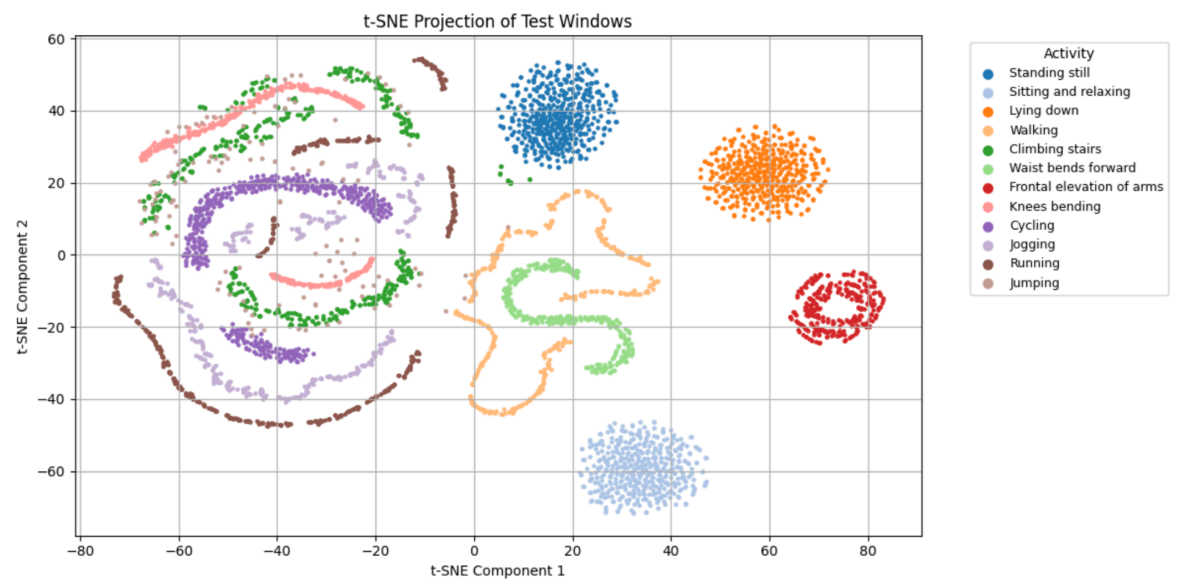
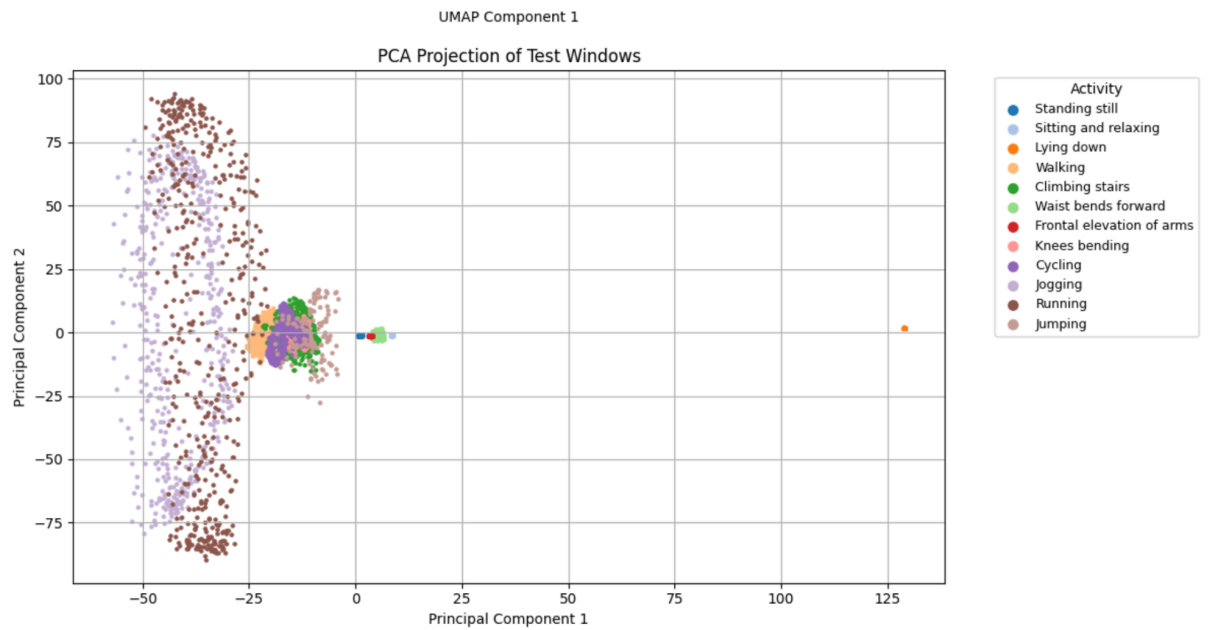
Part 2:

1. Training and validation loss curves.

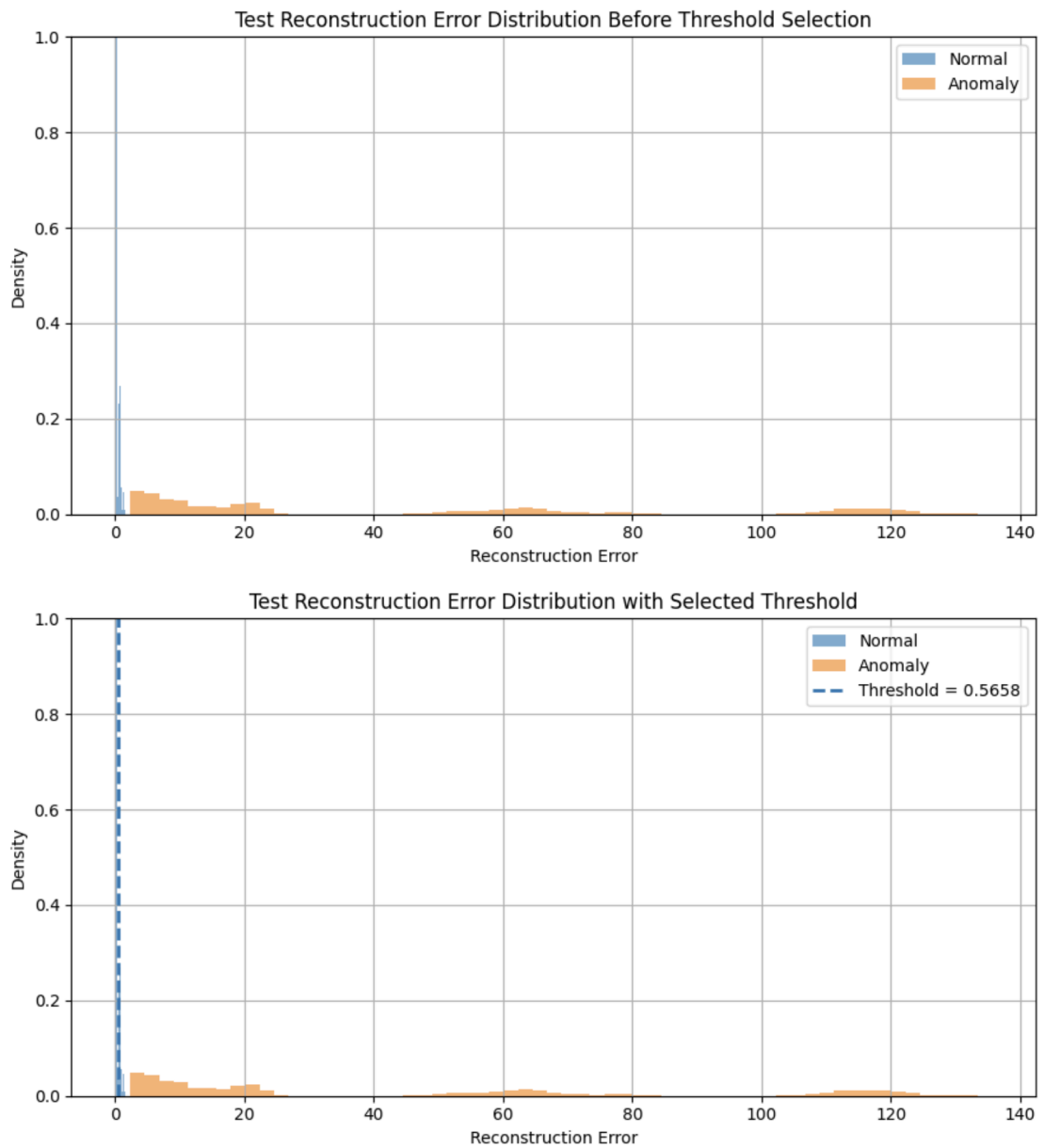


2. PCA, t-SNE, and UMAP visualizations.



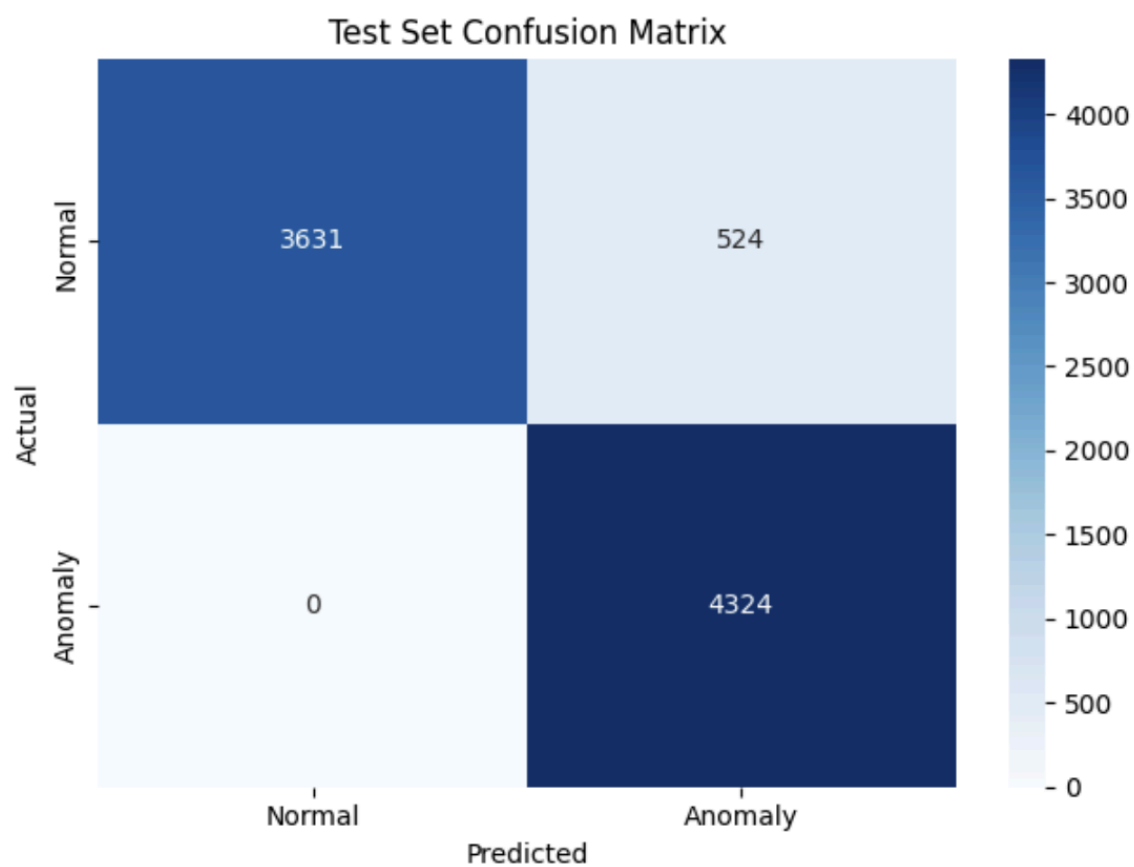
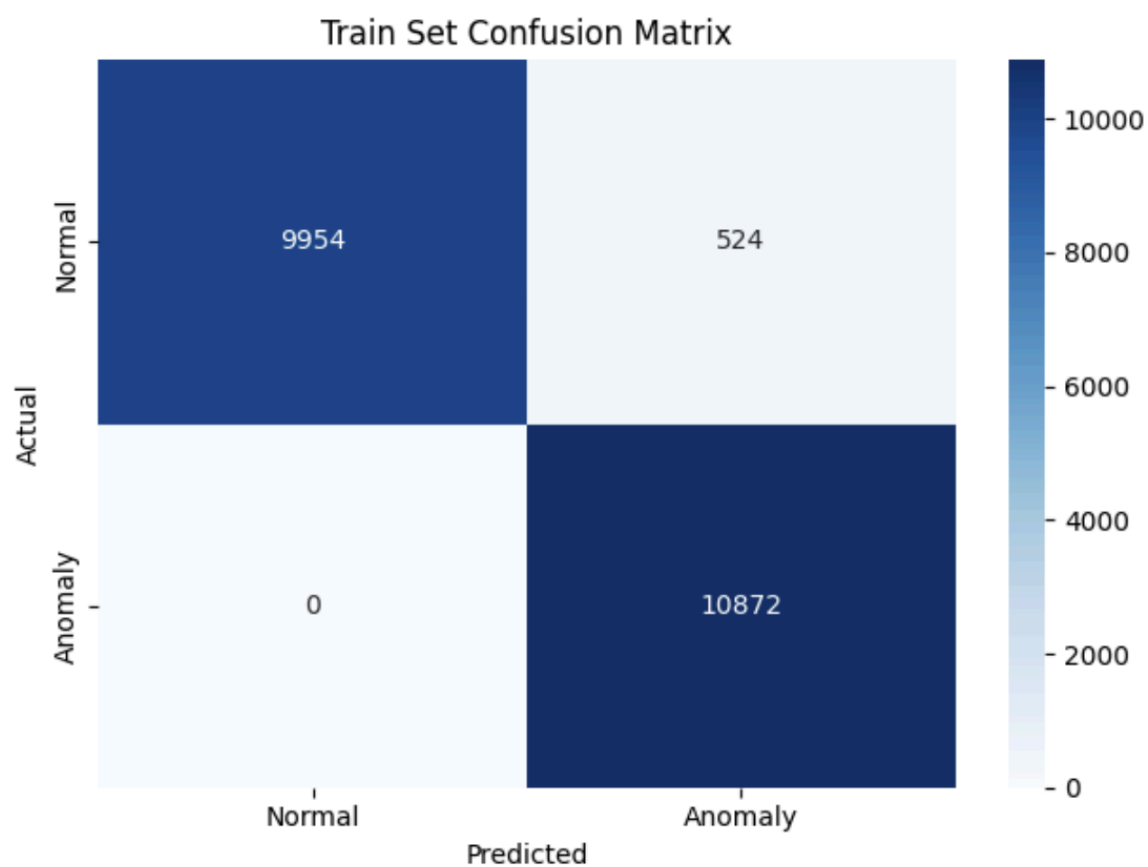


3. Reconstruction error distribution and selected threshold.



4. Confusion matrix and classification report.

Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.95	0.97	10478
Anomaly (1)	0.95	1.00	0.98	10872
accuracy			0.98	21350
macro avg	0.98	0.97	0.98	21350
weighted avg	0.98	0.98	0.98	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.87	0.93	4155
Anomaly (1)	0.89	1.00	0.94	4324
accuracy			0.94	8479
macro avg	0.95	0.94	0.94	8479
weighted avg	0.94	0.94	0.94	8479



5. Arduino serial monitor output showing reconstruction error and normal/anomaly detection result.

```
22:11:35.509 -> Result: Normal activity
22:11:35.543 -> Reconstruction error: 0.055378
22:11:35.543 -> Result: Normal activity
22:11:35.576 -> Reconstruction error: 0.043422
22:11:35.576 -> Result: Normal activity
22:11:35.610 -> Reconstruction error: 0.043678
22:11:35.610 -> Result: Normal activity
22:11:35.642 -> Reconstruction error: 0.043899
22:11:35.642 -> Result: Normal activity
22:11:35.642 -> Reconstruction error: 0.044070
22:11:35.642 -> Result: Normal activity
22:11:35.674 -> Reconstruction error: 0.044167
22:11:35.674 -> Result: Normal activity
22:11:35.708 -> Reconstruction error: 0.045847
22:11:35.708 -> Result: Normal activity
22:11:35.743 -> Reconstruction error: 0.046175
22:11:35.743 -> Result: Normal activity
22:11:35.775 -> Reconstruction error: 0.046318
22:11:35.775 -> Result: Normal activity
22:11:35.808 -> Reconstruction error: 0.048045
22:11:35.808 -> Result: Normal activity
22:11:35.808 -> Reconstruction error: 0.046861
22:11:35.808 -> Result: Normal activity
22:11:35.842 -> Reconstruction error: 0.043139
22:11:35.842 -> Result: Normal activity
22:11:35.876 -> Reconstruction error: 0.043395
22:11:35.876 -> Result: Normal activity
22:11:35.909 -> Reconstruction error: 0.043714
22:11:35.910 -> Result: Normal activity
22:11:35.942 -> Reconstruction error: 0.044112
22:11:35.942 -> Result: Normal activity
22:11:35.974 -> Reconstruction error: 0.044423
22:11:35.974 -> Result: Normal activity
22:11:35.974 -> Reconstruction error: 0.044703
22:11:35.974 -> Result: Normal activity
```

6. Short answers to the discussion questions listed in the submission guidelines.

Questions:

- What threshold did you choose for anomaly detection, and why?

I selected the anomaly detection threshold using the 95th percentile of the normal training reconstruction errors. I assumed normal samples should produce relatively small reconstruction errors because the autoencoder was trained only on normal activity data. I treated samples with reconstruction error above this threshold as anomalies.

- How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder learns to reconstruct normal activity patterns. When normal activity is given as input, the reconstructed output is usually very similar to the original input, resulting in low reconstruction error. For anomalous activity, the autoencoder cannot accurately reconstruct unfamiliar patterns, so it produces higher reconstruction error values.

- What information from the Python notebook must be preserved when deploying the model to Arduino?

The trained TensorFlow Lite model, quantization settings, preprocessing steps, input dimensions, and reconstruction-error threshold should be preserved.

- Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The Arduino sketch must use the same preprocessing steps as the notebook, so that the model was trained using a specific input format and scaling method.

- What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The main limitations include limited memory and power on microcontroller. These limitations restrict model complexity and may reduce detection accuracy.